AI-DRIVEN THREAT DETECTION & SIMULATION ENGINE

Hack Malenadu '26 | Cybersecurity Track | Problem Statement 3

Complete Project Blueprint

Idea Phase Submission Document

1. Executive Summary

This document presents a complete build blueprint for an AI-driven Threat Detection & Simulation Engine built for Hack Malenadu '26. The solution goes beyond traditional rule-based SIEM tools by combining a trained machine learning model with a live Docker-based attack simulation environment, giving SOC analysts real-time, explainable alerts and dynamically generated prevention playbooks.

The core innovation is a two-container Docker architecture: an Attacker container that generates realistic malicious traffic (brute force, lateral movement, data exfiltration, C2 beaconing) against a Victim/User container that runs our detection stack. Every event captured is fed through a trained ML model that classifies threats, assigns severity, and auto-generates a MITRE ATT&CK-mapped playbook — all rendered in a live SOC-style dashboard.

2. Problem Analysis & Our Approach

2.1 Core Challenges

- Security data is high-volume, high-noise, and adversarial — attackers deliberately craft patterns to evade detection
- Traditional rule-based systems catch only known patterns and overwhelm analysts with undifferentiated alerts
- Detection without explanation is useless to a real SOC analyst — context and next steps matter as much as the alert itself
- A single-layer alert is noise; the same behavior across network + endpoint layers simultaneously is a high-confidence incident

2.2 Our Approach

We address these challenges with three pillars:

- **Pillar 1:** Train Once, Detect Always
  - Train an ML classifier on the CICIDS 2017 dataset covering all four required threat categories
  - The trained model is embedded in the detection agent running inside the user container
- **Pillar 2:** Simulate to Validate
  - A dedicated attacker Docker container launches scripted attacks against the user container
  - This proves the system works on real traffic, not just clean benchmark data
- **Pillar 3:** Explain and Respond
  - Every alert includes a plain-English reason, MITRE ATT&CK mapping, confidence score, and a dynamically generated prevention playbook

3. Technical Architecture

3.1 High-Level Architecture Overview

The system is composed of four major layers that operate in a pipeline:

Layer	Component	Responsibility
Layer 1	Log Ingestion & Normalization	Collect raw logs from network + endpoint sources, normalize into unified schema
Layer 2	AI Threat Detection Engine	Classify events using trained ML model, assign confidence & severity
Layer 3	Correlation & Alert Engine	Cross-layer correlation, MITRE mapping, dedup, incident grouping
Layer 4	Dashboard & Playbook Generator	Live SOC dashboard, plain-English alerts, dynamic playbooks

3.2 Docker Architecture: Attacker vs. User Container

The foundation of our simulation environment is a two-container Docker setup orchestrated by Docker Compose:

Attacker Container

- Base image: Kali Linux (kalilinux/kali-rolling)
- Runs scripted attack scenarios using Python + standard offensive tools (nmap, hydra, hping3, netcat, curl)
- Attack scripts are fully parameterized and timed — each attack runs at a specified interval to mimic realistic adversary behavior
- Includes a false-positive generator: a legitimate admin script performing bulk file transfers that superficially resembles exfiltration

User / Victim Container

- Base image: Ubuntu 22.04 with Python 3.11
- Runs the Detection Agent: a Python process capturing network flows (via Scapy/tcpdump) and endpoint logs (via auditd/psutil)
- Events are streamed in real time to the ML inference engine
- Alerts are pushed to the SOC Dashboard via a FastAPI WebSocket backend

Shared Network

- Both containers share a Docker bridge network (threat_net) simulating a real LAN

- Traffic between containers is captured and analyzed at packet level

3.3 Data Flow Diagram

Below is the step-by-step data flow from attack generation to dashboard alert:

Step	Actor	Action	Output
1	Attacker Container	Launches attack script (e.g., brute force SSH)	Raw TCP/IP packets on threat_net
2	Detection Agent	Captures packets via Scapy, extracts flow features	Normalized event JSON
3	Detection Agent	Reads auditd logs for process & file activity	Endpoint event JSON
4	Normalization Layer	Merges network + endpoint events into unified schema	Correlated event batch
5	ML Inference Engine	Runs batch through trained RandomForest/XGBoost model	Label + confidence score
6	Alert Engine	Applies cross-layer correlation rules, maps to MITRE	High-confidence incident
7	Playbook Generator	Selects dynamic playbook template for incident type	Actionable response steps
8	FastAPI Backend	Pushes incident + playbook over WebSocket	JSON alert payload
9	React Dashboard	Renders live incident card with severity, reason, playbook	SOC analyst view

4. AI / ML Model: Training & Inference

4.1 Dataset

- Primary dataset: CICIDS 2017 (Canadian Institute for Cybersecurity Intrusion Detection System)
- Contains labeled network flows for: Benign, Brute Force (FTP/SSH), Port Scan, Botnet (C2), Web Attack, Infiltration, DDoS
- We use four subsets mapped to the four required threat categories:
  - Brute Force: Monday-WorkingHours, Tuesday — SSH/FTP Brute Force captures
  - Lateral Movement: Infiltration captures (Thursday)
  - Data Exfiltration: Infiltration + Web Attack captures with high outbound bytes
  - C2 Beaconing: Botnet captures — ISCX Botnet dataset subset for periodic beaconing patterns
- Benign traffic included in all files to train the model to handle false positives correctly

4.2 Feature Engineering

CICIDS 2017 already includes 78 pre-computed flow features. We use a curated subset of 30 high-signal features:

Feature Group	Features Used
Flow Duration	flow_duration, flow_bytes_per_sec, flow_packets_per_sec
Packet Stats	total_fwd_packets, total_bwd_packets, fwd_packet_len_mean, bwd_packet_len_mean
Flag Counts	fin_flag_count, syn_flag_count, rst_flag_count, psh_flag_count, ack_flag_count
IAT (Inter-Arrival Time)	flow_iat_mean, flow_iat_std, fwd_iat_total, bwd_iat_total
Subflow Stats	subflow_fwd_packets, subflow_bwd_packets, subflow_fwd_bytes, subflow_bwd_bytes
Activity	active_mean, idle_mean, init_win_bytes_fwd, init_win_bytes_bwd

4.3 Model Selection & Training

- Primary model: XGBoost Classifier (gradient-boosted trees — high accuracy on tabular security data, fast inference)
- Backup model: Random Forest (used for ensemble voting on ambiguous cases)
- Training pipeline (scikit-learn):
  - 1. Load and concatenate CICIDS CSV files
  - 2. Drop NaN/Inf values, normalize with StandardScaler
  - 3. SMOTE oversampling for minority classes (lateral movement is rare)

- 4. 80/20 train-test split, stratified by class
- 5. Train XGBoost with class_weight='balanced'
- 6. Evaluate: target F1 > 0.92 per class
- 7. Export model with joblib (.pkl) + feature list JSON

4.4 Confidence Score & Severity Mapping

Confidence Range	Severity Level	Action
0.90 - 1.00	CRITICAL	Immediate alert + auto-playbook + SOC notification
0.75 - 0.89	HIGH	Alert + playbook + analyst review required
0.55 - 0.74	MEDIUM	Alert with context + analyst investigation
0.40 - 0.54	LOW	Log only + flag for periodic review
< 0.40	BENIGN / FP	Suppress alert, log as false positive candidate

4.5 Explainability

- SHAP (SHapley Additive exPlanations) values computed per prediction
- Top 3 contributing features are rendered as plain-English reasons on the dashboard

Example output: "Alert: Brute Force (HIGH, 94% confidence). Reason: 847 failed auth attempts from 192.168.1.50 in 60s, SYN flood pattern detected, RST flag ratio 0.87. MITRE: T1110.001 — Password Guessing."

5. Attack Simulation Scenarios

The attacker container runs four scripted attack scenarios simultaneously (two are active by default in the demo, matching the problem statement requirement). Each attack is designed to generate realistic traffic patterns matching the CICIDS feature distribution.

5.1 Threat Category 1: Brute Force / Credential Stuffing

Attribute	Detail
MITRE ATT&CK	T1110.001 — Brute Force: Password Guessing
Tool Used	Hydra (SSH brute force) + custom HTTP login flood script
Target	Port 22 (SSH) and /api/login endpoint on user container
Pattern	847 requests in 60s from single IP, then distributed across 5 IPs
Detection Signal	High SYN count, RST ratio > 0.8, repeated 401/403 HTTP responses
False Positive Seed	Legitimate DevOps pipeline running 12 SSH key auth attempts — should NOT alert

5.2 Threat Category 2: Lateral Movement

Attribute	Detail
MITRE ATT&CK	T1021.002 — Remote Services: SMB/Windows Admin Shares
Tool Used	Custom Python script simulating internal port scanning + WMI-like calls
Pattern	After initial SSH compromise, scan 10 internal IPs on ports 135,445,3389
Detection Signal	Unusual internal-to-internal traffic, new process spawned under non-admin user
Endpoint Signal	auditd: execve() calls for net, whoami, ipconfig from compromised PID
Cross-Layer Trigger	Network scan pattern + endpoint process anomaly = HIGH confidence incident

5.3 Threat Category 3: Data Exfiltration

Attribute	Detail
MITRE ATT&CK	T1048.003 — Exfiltration Over Alternative Protocol
Tool Used	curl script transferring 500MB+ in repeated POST requests to external IP
Pattern	Outbound bytes > 100MB/flow, destination is non-whitelisted external IP

Attribute	Detail
Detection Signal	flow_bytes_per_sec spike, high bwd_packet_len_mean, unusual destination
FALSE POSITIVE INCLUDED	Admin backup script transferring 80MB to S3 — model must classify as BENIGN
Distinguishing Feature	S3 destination is whitelisted; time-of-day is business hours; process is backup.sh

5.4 Threat Category 4: C2 Beaconing

Attribute	Detail
MITRE ATT&CK	T1071.001 — Application Layer Protocol: Web Protocols
Tool Used	Python beacon script: sends 64-byte HTTP GET to external IP every 30s
Pattern	Periodic, low-volume, regular interval connections to external non-CDN IP
Detection Signal	flow_iat_std very low (regular interval), small packet size, repeated external dest
Endpoint Signal	Unknown process with no parent making outbound connections
Severity	CRITICAL — C2 implies active compromise and command channel

6. Technology Stack

6.1 Complete Tech Stack Reference

Category	Technology	Version	Purpose
ML / AI	XGBoost	2.0+	Primary threat classifier
ML / AI	scikit-learn	1.4+	Preprocessing, Random Forest, pipeline
ML / AI	SHAP	0.44+	Explainability — feature contribution per alert
ML / AI	imbalanced-learn	0.12+	SMOTE for minority class oversampling
ML / AI	pandas / numpy	Latest	Data loading, feature engineering
Detection Agent	Python 3.11	—	Core language for agent and backend
Detection Agent	Scapy	2.5+	Packet capture and flow feature extraction
Detection Agent	psutil	5.9+	Process and system monitoring
Detection Agent	auditd	System	Kernel-level endpoint event logging (Linux)
Backend API	FastAPI	0.110+	REST API + WebSocket for real-time alerts
Backend API	Uvicorn	0.29+	ASGI server
Backend API	Redis	7.2	Event queue and pub/sub for alert streaming
Frontend	React 18	18.x	SOC dashboard UI
Frontend	Recharts	2.x	Real-time charts (threat timeline, severity dist.)
Frontend	TailwindCSS	3.x	Styling
Frontend	Socket.IO Client	4.x	WebSocket connection to backend
Infrastructure	Docker	25+	Container runtime
Infrastructure	Docker Compose	v2	Multi-container orchestration
Infrastructure	Kali Linux	Rolling	Attacker container base image
Infrastructure	Ubuntu 22.04	LTS	User/victim container base image
Attack Tools	Hydra	9.4	Brute force simulation
Attack Tools	hping3	3.0	SYN flood / packet crafting
Attack Tools	nmap	7.94	Port scanning for lateral movement simulation
Attack Tools	netcat	1.10	C2 channel simulation
Dataset	CICIDS 2017	—	Primary training dataset (publicly available)

Category	Technology	Version	Purpose
Dataset	UNSW-NB15	—	Supplementary for C2 and exfiltration patterns

7. Complete Folder Structure

```
threat-detection-engine/
├── docker-compose.yml            # Orchestrates attacker + user containers
├── .env                          # Environment variables (ports, thresholds)
├── README.md
│
├── attacker/                     # ATTACKER CONTAINER
│   ├── Dockerfile
│   ├── requirements.txt
│   └── scripts/
│       ├── run_all_attacks.py    # Master orchestrator — runs all attacks
│       ├── brute_force.py        # Hydra SSH + HTTP login flood
│       ├── lateral_movement.py   # Internal port scan + pivot simulation
│       ├── exfiltration.py       # Large outbound transfer simulation
│       ├── c2_beacon.py          # Periodic 64-byte GET beacon
│       └── false_positive.py     # Legit admin backup transfer (FP seed)
│
├── user/                         # USER / VICTIM CONTAINER
│   ├── Dockerfile
│   ├── requirements.txt
│   └── agent/
│       ├── main.py               # Entry point — starts all agent threads
│       ├── capture/
│       │   ├── network_capture.py   # Scapy packet capture + flow extraction
│       │   └── endpoint_capture.py  # auditd + psutil process monitoring
│       ├── normalization/
│       │   ├── schema.py            # Unified event schema (Pydantic model)
│       │   └── normalizer.py        # Merges network + endpoint events
│       ├── inference/
│       │   ├── model_loader.py      # Loads .pkl model + scaler
│       │   ├── predictor.py         # Batch inference + SHAP explanation
│       │   └── models/              # Trained model artifacts
│       │       ├── xgboost_model.pkl
│       │       ├── scaler.pkl
│       │       └── feature_list.json
│       ├── correlation/
│       │   ├── correlator.py        # Cross-layer event correlation rules
│       │   └── mitre_mapper.py      # Maps threat class to MITRE ATT&CK ID
│       ├── playbook/
│       │   ├── generator.py         # Dynamic playbook builder
│       │   └── templates/           # Playbook templates per threat type
│       │       ├── brute_force.json
│       │       ├── lateral_movement.json
│       │       ├── exfiltration.json
│       │       └── c2_beacon.json
│       └── api/
```

```
│           ├── server.py            # FastAPI app with WebSocket endpoint
│           └── alert_schema.py      # Alert payload Pydantic model
│
├── ml_training/                    # OFFLINE ML TRAINING (run before demo)
│   ├── requirements.txt
│   ├── download_dataset.py         # Downloads CICIDS 2017 from UNB mirror
│   ├── preprocess.py               # Clean, normalize, SMOTE, split
│   ├── train.py                    # Train XGBoost + Random Forest
│   ├── evaluate.py                 # Classification report, confusion matrix
│   ├── export_model.py             # Saves .pkl artifacts to
user/agent/inference/models/
│   └── data/                       # Raw + processed CICIDS CSVs (gitignored)
│       ├── raw/
│       └── processed/
│
├── dashboard/                      # REACT SOC DASHBOARD
│   ├── package.json
│   ├── tailwind.config.js
│   ├── public/
│   └── src/
│       ├── App.jsx                 # Main layout
│       ├── components/
│       │   ├── IncidentFeed.jsx    # Real-time alert list
│       │   ├── IncidentCard.jsx    # Single alert with severity + playbook
│       │   ├── ThreatTimeline.jsx# Recharts timeline of events
│       │   ├── SeverityGauge.jsx # Live severity distribution donut chart
│       │   ├── MitrePanel.jsx     # MITRE ATT&CK technique cards
│       │   └── PlaybookDrawer.jsx# Slide-out playbook panel
│       └── hooks/
│           └── useAlertStream.js # WebSocket connection hook
│
└── docs/                           # DOCUMENTATION
    ├── architecture_diagram.png
    ├── project_blueprint.docx      # This document
    └── demo_script.md              # Step-by-step hackathon demo guide
```

8. Detection & Correlation Logic

8.1 Unified Event Schema

Every event — regardless of source — is normalized into this schema before inference:

Field	Type	Source	Description
event_id	UUID	Generated	Unique event identifier
timestamp	ISO8601	Capture time	When the event was observed
layer	Enum	network/endpoint/application	Signal layer source
src_ip	String	Network capture	Source IP address
dst_ip	String	Network capture	Destination IP address
src_port	Int	Network capture	Source port
dst_port	Int	Network capture	Destination port
protocol	String	Network capture	TCP/UDP/ICMP
flow_duration	Float	Derived	Flow duration in microseconds
flow_bytes_per_sec	Float	Derived	Bytes per second
flag_counts	Dict	Derived	SYN/RST/FIN/ACK counts
process_name	String	Endpoint agent	Executing process name
parent_pid	Int	Endpoint agent	Parent process ID
user	String	Endpoint agent	User executing the process
file_access	List	Endpoint agent	Files accessed by process
confidence	Float	ML model	Prediction confidence 0-1
threat_class	String	ML model	Predicted threat category
severity	Enum	Derived	LOW/MEDIUM/HIGH/CRITICAL
mitre_id	String	Mapper	MITRE ATT&CK technique ID
explanation	String	SHAP	Plain-English reason for alert

8.2 Cross-Layer Correlation Rules

- Rule 1 (Brute Force Confirmation): Network SYN flood from IP X AND endpoint log shows >50 failed auth PAM events from same IP within 60s → CRITICAL
- Rule 2 (Lateral Movement Confirmation): Internal port scan detected on network layer AND auditd shows net/whoami/ipconfig execution from same host → CRITICAL

- Rule 3 (Exfiltration vs. FP): High outbound bytes to external IP. IF destination in whitelist AND process is known backup agent AND time within business hours → BENIGN. Otherwise → HIGH/CRITICAL
- Rule 4 (C2 Detection): Periodic low-volume connections (IAT std < 5s) to external non-CDN IP AND process has no documented parent → CRITICAL

9. Dynamic Playbook Generation

For each high-confidence incident, the playbook generator selects the appropriate template and fills in dynamic context (source IP, process name, affected host, timestamp) to produce an analyst-ready response guide.

9.1 Example: Brute Force Playbook Output

Step	Action	Command / Detail
1 — Contain	Block attacker IP at firewall	iptables -A INPUT -s {src_ip} -j DROP
2 — Contain	Lock affected account temporarily	usermod -L {targeted_user}
3 — Investigate	Dump auth log for IP	grep {src_ip} /var/log/auth.log | tail -100
4 — Investigate	Check for successful logins	last | grep {src_ip}
5 — Eradicate	Force password reset for targeted accounts	passwd -e {targeted_user}
6 — Harden	Enable fail2ban for SSH	systemctl enable fail2ban && systemctl start fail2ban
7 — Harden	Restrict SSH to key-based auth	Set PasswordAuthentication no in /etc/ssh/sshd_config
8 — Report	Document incident in SIEM	Log MITRE T1110.001 event with evidence bundle

9.2 Example: C2 Beacon Playbook Output

Step	Action	Command / Detail
1 — Isolate	Quarantine host from network immediately	docker network disconnect threat_net {container}
2 — Contain	Block C2 IP at perimeter	iptables -A OUTPUT -d {c2_ip} -j DROP
3 — Investigate	Identify beaconing process	ss -tp | grep {c2_ip} && ps aux | grep {pid}
4 — Investigate	Dump process memory for IOCs	gcore {pid} && strings core.{pid} | grep -E 'http|cmd'

Step	Action	Command / Detail
5 — Eradicate	Kill malicious process, remove persistence	kill -9 {pid} && check crontab, rc.local, systemd
6 — Hunt	Search for lateral movement indicators	Run lateral movement detection on all internal hosts
7 — Report	File IR report with MITRE T1071.001	Include beacon interval, C2 IP, process name, IOCs

10. SOC Dashboard Design

10.1 Dashboard Components

Component	Description	Data Source
Incident Feed	Real-time scrolling list of alerts with severity badge, timestamp, threat class	WebSocket stream
Incident Card	Expandable card: threat class, confidence %, MITRE ID, plain-English explanation, affected IPs	ML inference output
Threat Timeline	Recharts line chart — events per minute over last 15 minutes, colored by severity	Redis time-series
Severity Gauge	Donut chart showing distribution of LOW/MEDIUM/HIGH/CRITICAL over current session	Redis counters
MITRE Panel	Grid of detected ATT&CK techniques with tactic, technique ID, and detection count	Correlation engine
Playbook Drawer	Slide-out side panel with step-by-step response guide for selected incident	Playbook generator
False Positive Flag	Yellow badge on cards where model confidence < 0.55 or whitelist rule matched	Correlation engine
Event Rate Meter	Live events/second counter — alerts if > 500 eps (stress test indicator)	Detection agent

10.2 Performance: 500+ Events/Second Handling

- Detection agent batches events in 500ms windows before inference (reduces model call overhead)
- Redis Streams used as the event queue — supports 100,000+ events/second at single-node
- FastAPI with async WebSocket handlers — non-blocking push to dashboard
- Dashboard uses virtual scrolling (react-virtualized) to render thousands of incidents without lag

11. Build & Run Instructions

Step 1: Train the ML Model (offline, before hackathon)

```
cd ml_training
pip install -r requirements.txt
python download_dataset.py      # Downloads CICIDS 2017 (~1GB)
python preprocess.py            # Clean, SMOTE, split
python train.py                 # Trains XGBoost + RF
python evaluate.py              # Prints classification report
python export_model.py          # Copies .pkl to user/agent/inference/models/
```

Step 2: Build Docker Images

```
docker compose build            # Builds attacker + user images
```

Step 3: Start the Dashboard

```
cd dashboard && npm install && npm start
```

Step 4: Launch the Simulation

```
docker compose up               # Starts both containers
# Attacker begins attack sequence after 10s warm-up
# Dashboard at http://localhost:3000
# API at http://localhost:8000/docs
```

Step 5: Demo Flow

- T+0:00 — System starts, baseline benign traffic visible on dashboard
- T+0:30 — Brute force attack begins (SSH hydra) — CRITICAL alert fires within 15s
- T+1:00 — C2 beacon activates (running simultaneously) — CRITICAL alert with T1071.001
- T+1:30 — Admin backup script runs — model correctly classifies as BENIGN (false positive demo)
- T+2:00 — Lateral movement scan begins — cross-layer correlation fires HIGH incident
- T+3:00 — Exfiltration attempt — HIGH alert fires; admin backup correctly suppressed
- Open Playbook Drawer for any incident — show dynamic response steps

12. Suggested Improvements & Stretch Goals

The following section proposes enhancements beyond the minimum bar that can significantly elevate the solution's real-world applicability and hackathon impact:

12.1 Threat Hunting Mode (Proactive)

- Instead of waiting for alerts, implement a scheduled threat hunt that queries the Redis event history for low-confidence patterns that haven't triggered alerts
- Cluster recent benign-classified events with DBSCAN — sudden clusters of similar 'benign' events may indicate a slow-burn attack evading the threshold
- Implementation: Add a /api/hunt endpoint that runs DBSCAN on last 1000 events and returns anomaly clusters

12.2 Adversarial Robustness Testing

- Add a mode where the attacker container deliberately tries to evade the model by modifying packet timing and sizes to match benign traffic distributions
- This validates that the model generalizes beyond clean benchmark data — a real differentiator in a hackathon context
- Track evasion success rate and show it on dashboard — demonstrates awareness of adversarial ML

12.3 LLM-Powered Playbook Narration

- Integrate Claude API (claude-sonnet-4) to generate a natural-language incident narrative from structured alert data
- Example: Instead of a bulleted playbook, produce a paragraph: 'At 14:32, attacker 192.168.1.50 launched a distributed brute force against your SSH service. 847 failed attempts were recorded in 60 seconds. The attack was stopped before any successful login. Recommended immediate actions: ...'
- This transforms static playbooks into dynamic, context-aware analyst briefings

12.4 Application Layer (Layer 3) Integration

- Add a lightweight Flask app in the user container serving a mock /api/login endpoint
- Log all HTTP requests (method, endpoint, status, user-agent, payload size) and feed into the normalization layer
- This completes the three-layer ingestion requirement and enables web attack detection (SQL injection attempts, web scraping) as a fifth threat category

12.5 MITRE ATT&CK Navigator Export

- Generate a navigator-layer JSON file after each simulation session showing which ATT&CK techniques were detected
- This is a standard SOC deliverable and immediately demonstrates professional-grade output to judges

12.6 Alert Deduplication & Suppression

- Implement a 5-minute suppression window per (src_ip, threat_class) pair — prevents alert fatigue from repeated identical events
- Show suppression count on dashboard: '14 similar alerts suppressed — expand to view'

13. MITRE ATT&CK Mapping Summary

Threat Category	MITRE Tactic	Technique ID	Technique Name
Brute Force	Credential Access	T1110.001	Brute Force: Password Guessing
Brute Force (Distributed)	Credential Access	T1110.004	Brute Force: Credential Stuffing
Lateral Movement	Lateral Movement	T1021.002	Remote Services: SMB/Windows Admin Shares
Lateral Movement (Discovery)	Discovery	T1046	Network Service Discovery
Data Exfiltration	Exfiltration	T1048.003	Exfiltration Over Alternative Protocol
C2 Beaconing	Command & Control	T1071.001	Application Layer Protocol: Web Protocols
C2 (Timing)	Command & Control	T1571	Non-Standard Port
Endpoint Discovery	Discovery	T1057	Process Discovery

14. Hackathon Submission Checklist

Requirement	Status	Implementation
Signal layers (min 1)	2 layers	Network (Scapy) + Endpoint (auditd/psutil)
Threat categories (min 2)	4 categories	Brute Force, Lateral Movement, Exfiltration, C2
MITRE mapping	Yes	8 techniques mapped across 4 threat classes
Explainability	Yes	SHAP values → plain-English reasons per alert
Dynamic playbooks	Yes	Context-aware playbooks with actual IP/process/commands
Live dashboard	Yes	React SOC dashboard with WebSocket real-time stream
False positive included	Yes	Admin backup script correctly classified as BENIGN
500 eps handling	Yes	Batched inference + Redis Streams + async FastAPI
Two simultaneous attacks	Yes	Brute force + C2 beacon running in parallel
Architecture diagram	Yes	See docs/architecture_diagram.png
Tech stack documented	Yes	Section 6 of this document
Team details	Fill in	Add team name, member names, institution

— End of Document —

Hack Malenadu '26 | 36 Hours | Team Size: 2-4 | No tech stack restrictions